

Unitat 8.3

Control d'errors

Warnings en temps de compilació

Indiquen condicions/circumstàncies que no causen problemes de compilació. Per exemple: ús de codi PL/SQL deprecats.

Generats durant la compilació d'objectes PL/SQL.

Forma: PLW-nnnn

Warnings en temps de compilació

Categories:

- SEVERE: pot causar accions inesperades i/o resultats erronis.
 - Exemple: Problema d'aliasing amb els paràmetres
- PERFORMANCE: pot causar problemes de rendiment:
 - Exemple: Passar un VARCHAR2 a una columna NUMBER, a un INSERT.
- INFORMATION: No afecta el rendiment o la correctesa del codi, però si impacte negativament la seva mantenibilitat.
 - Exemple: codi inabastable (*unreachable code*).

Paràmetre de compilació PLSQL_WARNINGS

Permet:

- Activar o desactivar tots els *warnings*, o alguns en concret.
- Tractar *warnings* específics com errors, de manera que impedeixin la compilació dels objectes PL/SQL afectats.

Es pot establir a nivell de:

- Base de dades: `ALTER SYSTEM SET PLSQL_WARNINGS='ENABLE:ALL';`
- Sessió: `ALTER SESSION SET PLSQL_WARNINGS='ENABLE:ALL';`
- Unitat PL/SQL: `ALTER PROCEDURE loc_var COMPILE PLSQL_WARNINGS='ENABLE:PERFORMANCE'`

Warnings: example

```
create or replace FUNCTION S_GET_JOB_TITLE(p_first_name varchar2, p_last_name varchar2) RETURN VARCHAR2 IS
    vJobTitle EMPLOYEES.JOB_TITLE%TYPE;
BEGIN
    RETURN 'd';
    SELECT JOB_TITLE INTO vJobTitle FROM EMPLOYEES WHERE FIRST_NAME = p_first_name and LAST_NAME = p_last_name;
    RETURN vJobTitle;
END;
```

ION S_GET_JOB_TITLE > BEGIN > SELECT

mpiler - Log

Project: C:\Users\benna\AppData\Roaming\SQL Developer\system22.2.1.234.1810\o.sqldeveloper\projects\IdeConnections#CIDE+user.jpr

Function EXRECU.S_GET_JOB_TITLE@CIDE user



Warning(1,1): PLW-05018: unit S_GET_JOB_TITLE omitted optional AUTHID clause; default value DEFINER used



Warning(5,9): PLW-06002: Unreachable code

Excepcions

Errors en temps d'execució (*runtime error*).

Causats per:

- Errors de disseny
- Errors de codi
- Errors de *hardware*
- ...

Es gestionen a la part de control d'errors d'un bloc

Exception handlers

A la part de control d'errors.

Sintaxi: `WHEN nom_excepcio THEN ...`

El codi dins cada *handler* s'executa si ocorre l'excepció especificada.

OTHERS: qualsevol excepció.

```
BEGIN
    -- executable section
    ...
    -- exception-handling section
EXCEPTION
    WHEN e1 THEN
        -- exception_handler1
    WHEN e2 THEN
        -- exception_handler1
    WHEN OTHERS THEN
        -- other_exception_handler
END;
```

Exception handlers

Si una excepció no es gestiona dins el bloc, és propagarà als blocs externs.

Per exemple, si cream una funció *standalone* que no gestiona l'excepció `NO_DATA_FOUND` i aquest error ocorre quan cridam la funció des d'un bloc anònim, l'haurà de gestionar el bloc anònim o si no quedarà sense gestionar i aturarà l'execució del codi.

Tipus d'excepcions

Definides internament:

- Aixecades automàticament pel sistema.
- Tenen codi d'error però no nom (tot i que se n'hi pot assignar un).

Predefinides:

- Internament definides, amb un nom (ex: NO_DATA_FOUND).
- Llista: [PL/SQL Error Handling \(oracle.com\)](http://oracle.com)

Definides per l'usuari:

- Declarades per l'usuari a un bloc, subprograma o paquet.
- Aixecades/tirades explícitament.

Excepcions predefinides: exemple

```
DECLARE
    l_product_name varchar2(255);
BEGIN
    SELECT product_name INTO l_product_name
        FROM products WHERE category_id = 3;
END;
```

Script Output x

Task completed in 0,05 seconds

Error report -
ORA-01403: no s'ha trobat cap dada
ORA-06512: a line 4
01403. 00000 - "no data found"
*Cause: No data was found from the objects.
*Action: There was no data from the objects which may be due to end of fetch.

Sense exception handler

```
DECLARE
    l_product_name varchar2(255);
BEGIN
    SELECT product_name INTO l_product_name
        FROM products WHERE category_id = 3;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('No products for category 3');
END;
```

Script Output x

Task completed in 0,042 seconds

PL/SQL procedure successfully completed.

Dbms Output

Buffer Size: 20000

BBDD x

No products for category 3

Amb exception handler

Excepcions predefinides: exercici

Intenta dividir un nombre per zero guardant el resultat de l'expressió dins una variable NUMBER que hagi declarat anteriorment.

Gestiona l'excepció ZERO_DIVIDE i mostra un missatge indicant que l'operació és invàlida.

Excepcions definides internament

Per capturar excepcions definides internament, els hi hem d'assignar un nom.

Passes:

1. Declarar el nom d'excepció que utilitzarem:

```
nom_excepcio EXCEPTION;
```

2. Associar el codi de l'excepció internament definida amb el nom d'excepció anterior:

```
PRAGMA EXCEPTION_INIT (nom_excepcio, codi_error) ;
```

Excepcions definides per l'usuari

Declaració:

```
nom_excepcio EXCEPTION;
```

A la part declarativa d'un bloc anònim, subprograma o paquet.

Han de ser aixecades/tirades explícitament:

```
RAISE exception_name;
```

Excepcions definides per l'usuari: exemple

```
DECLARE
    NOT_FOUND exception;
    l_employee_id number := 44;

    FUNCTION get_employee_name(p_employee_id EMPLOYEES.EMPLOYEE_ID%TYPE)
    RETURN VARCHAR2 IS
        l_aux number;
        l_name varchar2(255);
    BEGIN
        SELECT count(*) INTO l_aux FROM employees
            WHERE employee_id = p_employee_id;
        IF (l_aux = 0) THEN
            RAISE NOT_FOUND;
        END IF;
        SELECT first_name INTO l_name FROM employees
            WHERE employee_id = p_employee_id;
        RETURN l_name;
    END;

BEGIN
    DBMS_OUTPUT.PUT_LINE(get_employee_name(l_employee_id));
EXCEPTION
    WHEN NOT_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('ERROR! Employee not found');
END;
```

```
DECLARE
    NOT_FOUND exception;
    l_employee_id number := 244;

    FUNCTION get_employee_name(p_employee_id EMPLOYEES.EMPLOYEE_ID%TYPE)
    RETURN VARCHAR2 IS
        l_aux number;
        l_name varchar2(255);
    BEGIN
        SELECT first_name INTO l_name FROM employees
            WHERE employee_id = p_employee_id;
        RETURN l_name;
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            RAISE NOT_FOUND;
    END;

BEGIN
    DBMS_OUTPUT.PUT_LINE(get_employee_name(l_employee_id));
EXCEPTION
    WHEN NOT_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('ERROR! Employee not found');
END;
```

Excepcions definides per l'usuari: Exercici

Crea una funció de cerca d'empleats que, donat un *varchar2*, miri si el *first_name* i *last_name* d'algun empleat coincideixen i en retorni l'ID. Utilitza LIKE. Si hi ha més d'un empleat coincident retorna el primer per ordre alfabètic.

Si no es troba cap empleat, aixeca una excepció que hagi definit anteriorment.

Després, crea un bloc anònim que cerqui un empleat no existent i gestiona l'excepció, mostrant un missatge "Employee not found".

Tipus d'excepcions

Categoria	Definer	Té codi d'error	Té nom	S'aixeca implícitament	S'aixeca explícitament
Definida internament	Sistema	Sempre	Si se li assigna.	Sí	Opcionalment
Predefinida	Sistema	Sempre	Sempre	Sí	Opcionalment
Definida per l'usuari	Usuari	Si se li assigna.	Sempre	No	Sempre

Exemples d'exceptions

- NO_DATA_FOUND:

- S'origina quan un SELECT no retorna dades.

- Exemple: `SELECT * FROM employees WHERE employee_id = 999;`

- TOO_MANY_ROWS:

- Originada quan s'intenta guardar més d'una fila dins un record/variable.

- Exemple:

```
DECLARE
    l_name VARCHAR2(255);
BEGIN
    SELECT first_name INTO l_name FROM employees WHERE employee_id IN (1,2);
END;
```

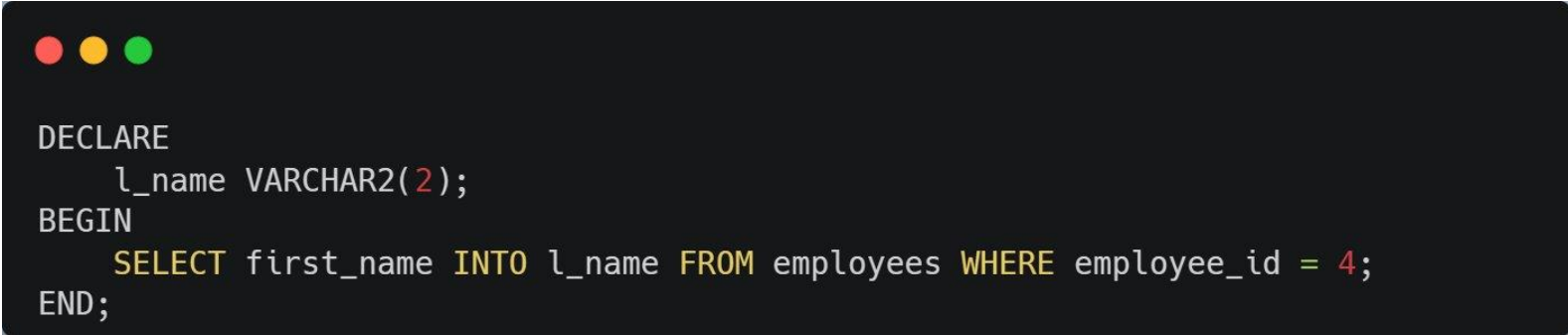
Exemples d'excepcions

- INVALID_NUMBER:

- S'origina quan falla la conversió d'un string a un nombre, perquè l'*string* no representa un número vàlid.
- Exemple:

- VALUE_ERROR:

- Originada quan ocorre un error aritmètic, de conversió, de truncatge o de mida.
- Exemple:



```
DECLARE
    l_name VARCHAR2(2);
BEGIN
    SELECT first_name INTO l_name FROM employees WHERE employee_id = 4;
END;
```

Excepcions de cursors

- `CURSOR_ALREADY_OPEN`:
 - Originada en intentar obrir un cursor que ja ha estat obert.
- `INVALID_CURSOR`:
 - S'origina quan es duu a terme una operació invàlida a un cursor.
 - Exemple: Tancar o treure dades d'un cursor no obert.

Propagació d'excepcions: exemple

```
DECLARE
    exception_dog EXCEPTION;

    PROCEDURE dog(in_param NUMBER) IS
    BEGIN
        RAISE exception_dog;
    END;

    PROCEDURE foo(in_param NUMBER) IS
    BEGIN
        dog(1);
    END;

BEGIN
    foo(55);
END;
```

The diagram consists of two orange arrows. The first arrow originates from the line 'dog(1);' within the 'foo' procedure and points to the error message. The second arrow originates from the line 'foo(55);' within the main 'BEGIN' block and also points to the same error message, indicating the call stack path of the exception.

06510. 00000 - "PL/SQL: unhandled user-defined exception"

*Cause: A user-defined exception was raised by PL/SQL code, but not handled.

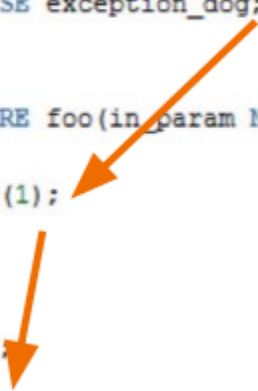
Propagació d'excepcions: exemple

```
DECLARE
    exception_dog EXCEPTION;

    PROCEDURE dog(in_param NUMBER) IS
    BEGIN
        RAISE exception_dog;
    END;

    PROCEDURE foo(in_param NUMBER) IS
    BEGIN
        dog(1);
    END;

BEGIN
    foo(55);
EXCEPTION
    WHEN exception_dog THEN DBMS_OUTPUT.PUT_LINE('Exception: Dog caught');
END;
```

Two orange arrows illustrate the flow of exception propagation. The first arrow starts at the 'dog(1);' line within the 'foo' procedure and points to the 'foo(55);' line in the main 'BEGIN' block. The second arrow starts at the 'foo(55);' line and points to the 'WHEN exception_dog THEN' line in the 'EXCEPTION' block, showing how the exception is caught at the outermost level.

Exception: Dog caught

Procedimiento PL/SQL terminado correctamente.

Execució després d'una excepció gestionada

Després de gestionar una excepció, l'execució del codi continua normalment al bloc pare d'on s'hagi produït:

```
FUNCTION get_division(in_dividend PLS_INTEGER,in_divider PLS_INTEGER)
RETURN PLS_INTEGER IS
    l_quotient PLS_INTEGER;
BEGIN
    l_quotient := in_dividend/in_divider;
    RETURN l_quotient;
EXCEPTION
    WHEN ZERO_DIVIDE THEN
        RETURN in_dividend; -- We keep the program running
END;

BEGIN
    g_val := get_division(5,0); -- get_division returns 5
    DBMS_OUTPUT.PUT_LINE(get_division(10,g_val));
END;
```

Execució després d'una excepció: exercici

Modifica els subprogrames del final del tema 8.2 per què, en cas de no trobar un empleat que compleixi amb les condicions, retornin (o posin dins els paràmetres OUT) "NA" i l'execució pugui seguir.

Executa els dos subprogrames (procediment i funció) amb FIRST_NAME i LAST_NAME inexistents, un darrera l'altre, i comprova que el programa segueix.